

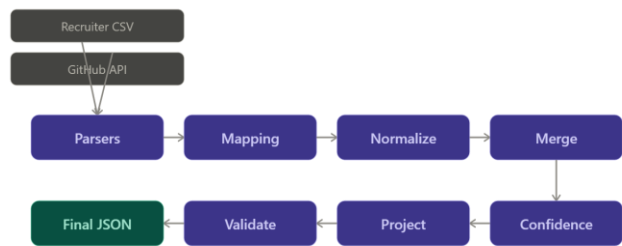
MULTI-SOURCE CANDIDATE DATA TRANSFORMER

Technical Design

1. Design Philosophy

A deterministic, explainable engine that converts heterogeneous candidate data into a single canonical profile. Wrong-but-confident values are treated as worse than honestly empty, so the normalizer returns None rather than fabricating data. Strict separation between parsing, normalization, merging, and projection ensures each stage has exactly one responsibility.

2. System Architecture



3. Canonical Schema

Canonical Model

A typed Candidate dataclass serves as the single internal representation. Nested structures (location, links, skills) remain dictionaries/lists to support flexible path-based projection while preserving schema consistency. Data is normalized to E.164 phone numbers, ISO-3166 country codes, and canonical skill names.

Fields

candidate_id • full_name • emails[] • phones[]
location{} • links{} • headline • years_experience
skills[] • experience[] • education[] • provenance{} • overall_confidence

Sources Implemented

Structured: Recruiter CSV

Unstructured: GitHub Public API (free-text bio, inferred skills from repository languages, and source-specific fields requiring canonical mapping).

4. Merge / Conflict-Resolution Policy

Field	Strategy	Rationale
full_name, headline, years_experience	Priority: CSV → GitHub	CSV preferred
emails, phones	Normalize → Union → Deduplicate	Preserve all contacts
location	First non-null (CSV priority order)	Deterministic
links	Merge by type	Independent link types
skills	Union + Confidence-Weighting	Multi-source corroboration

Merge decisions are deterministic. Every selected value is accompanied by field-level provenance {field, value, source, method}.

5. Confidence Strategy

Rule-based (no ML) for complete explainability. Each populated field contributes a fixed weight (e.g. full_name = 0.20, emails = 0.18, phones = 0.10, skills = 0.10), while supplementary fields (location, links, experience, education, etc.) contribute proportionally smaller weights. An additional +0.03 bonus is awarded when a field is corroborated by multiple sources, and a -0.15 penalty is applied for each missing required field (full_name, emails). Scores are clamped to [0,1] and returned with human-readable reasons for explainability.

6. Runtime Configuration

Runtime-Configurable Output:

- Field selection
- Field remapping (from)
- Per-field normalization
- Provenance / confidence toggles
- Missing-value policy (null, omit, error)
- Nested path support (links.github, skills[].name)

Only the configuration determines the output schema.

Code Snippet:

```
{
  "fields": [
    {
      "path": "primary_email",
      "from": "emails[0]",
      "required": true,
    },
    {
      "path": "phone",
      "from": "phones[0]",
      "normalize": "E164",
    },
    {
      "path": "skills",
      "from": "skills[].name",
      "normalize": "canonical"
    }
  ],
  "include_confidence": false,
  "on_missing": "omit"
}
```

7. Edge Cases

Scenario	Behaviour
Missing GitHub	Continue with CSV
API Failure	Log and continue
Invalid Email	Flag record
Conflicting values	Deterministic merge

8. Engineering Trade-offs

- Chose **GitHub over resume parsing** to focus on the transformation engine rather than **OCR/layout extraction**.
- Used one **typed canonical Candidate model** while keeping nested structures dictionary-based for **flexible projection**.
- Implemented a **CLI instead of a GUI**, aligning with the assignment's emphasis on **engine correctness** over interface polish.